实验 6-存储器实验

一、 实验准备

本次实验会在实验 3 中完成的串口通讯实验基础上进行。在之前的实验中我们已经能使用串口收发数据，本次实验需要你能够使用串口的数据与 FPGA 上的电路进行交互，并熟悉片上存储器的操作方式。在进行本次实验之前，你需要具备以下基础能力：

1. 熟悉串口模块的操作方式；
2. 熟悉 Xilinx 库 IP 使用方式。

本次实验会提供包装好的串口模块，你需要根据实验要求完成所需功能。

二、 实验目的

2.1 了解随机存取存储器 RAM 的原理；
2.2 掌握调用 Xilinx 库 IP(Block Memory Generator)实例化 RAM 的方法；

三、 实验任务

本次实验使用 Vivado 的 Block Memory Generator 模拟数据在存储器中的存取过程。实验使用单端口 ROM。初始化 ROM 存储器中的内容，通过串口的 RX 接收地址，将对应的存储器中内容读出来，并通过 TX 发送回 PC。实验原理如图 1 所示：

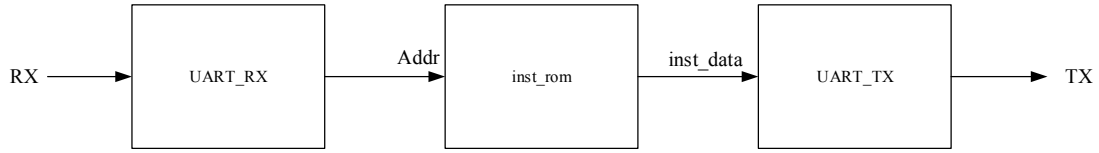
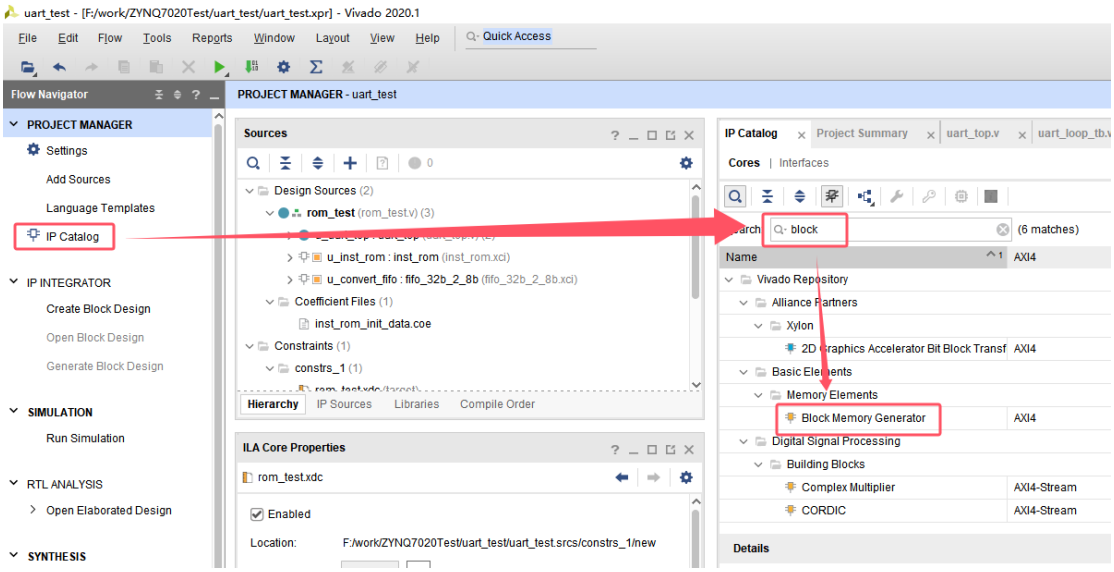


图 1 存储器实验原理图

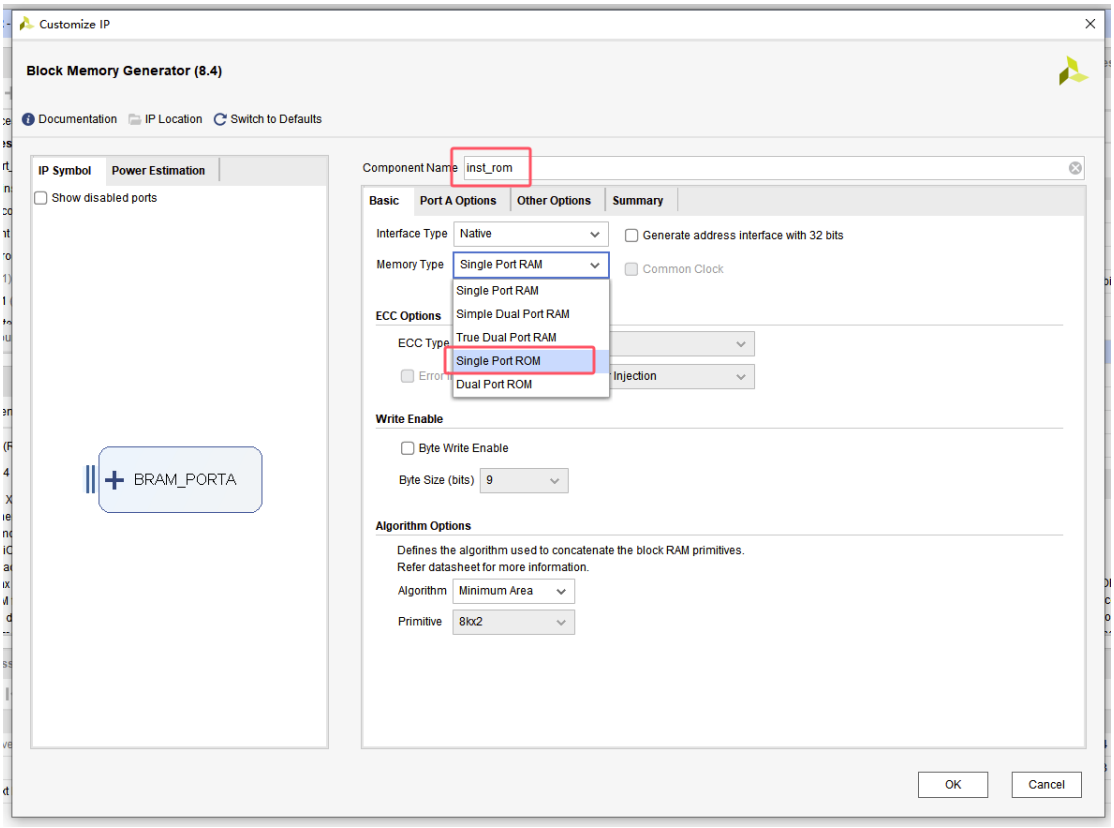
四、 实验步骤

1. 添加调用 rom IP 核

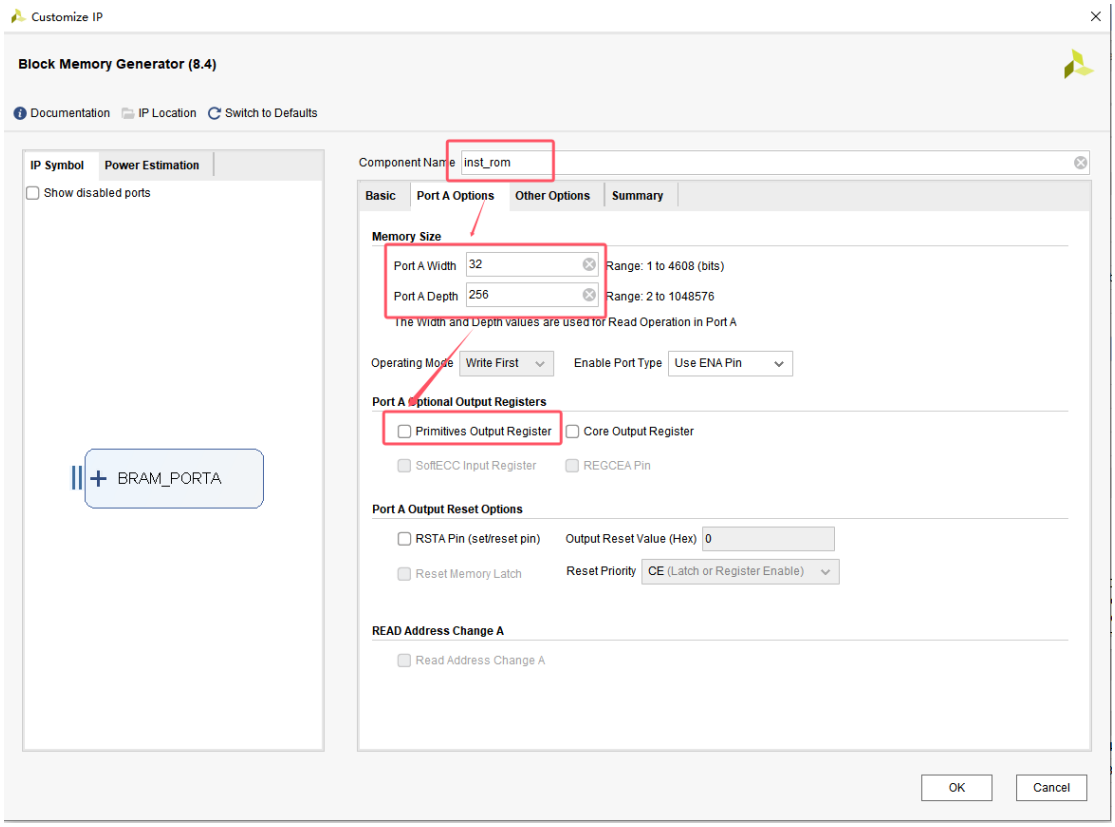
在 IP Catalog > Memories & Storage Elements > RAMs & ROMs> Block Memory Generator 目录底下，找到系统自带的 rom 核并添加。点击 Flow Navigator 中的 IP Catalog，打开窗口添加 IP 核。



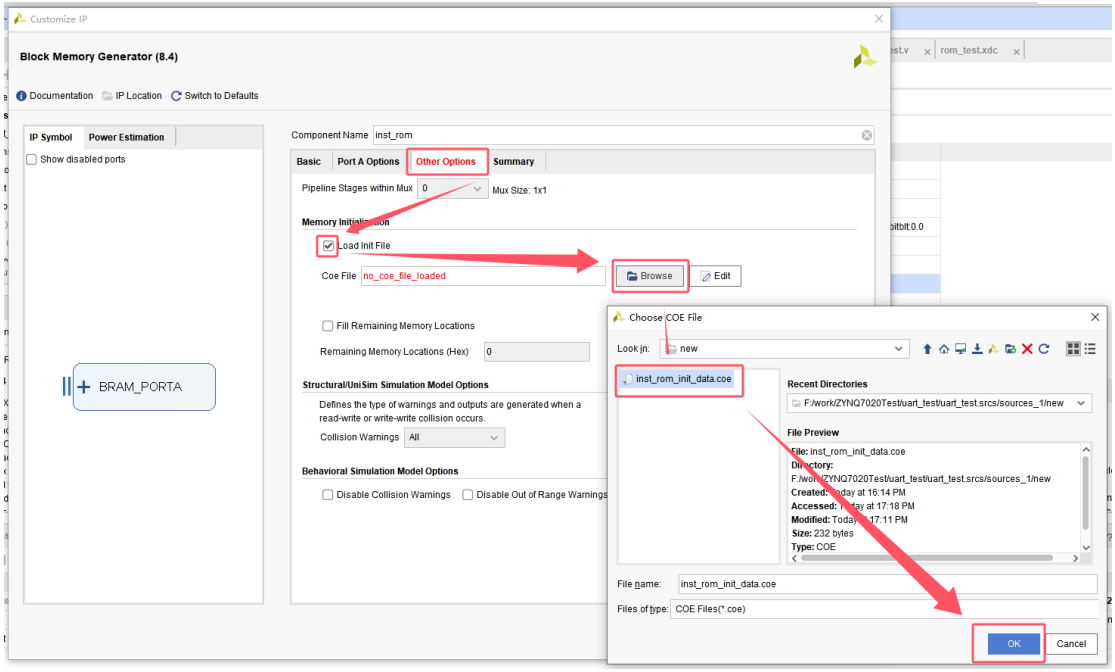
双击之后开始编辑 ROM 的功能：



Component Name 命名为：inst_rom，名字可以自己决定。基础信息选择 Native、单端口、ROM 不要 ECC 校验、最小面积算法。接着设置 Port A Option 的基本属性



数据宽度 32 位，深度要 256 个数据，地址线 8 根，去掉输出端的寄存器(会增加一拍的延时)。接着设置 option 选项属性：



在 Load Init File 项上打勾，点击 Browse，要你选择一个文件。这个文件就是你要放在 ROM 中的数据，文件后缀名为 coe，名称自己决定。你可以自己建一个文本文件，然后把后缀名改为.coe 即可。(文件中放入自己的学号或你想显示任意内容)。

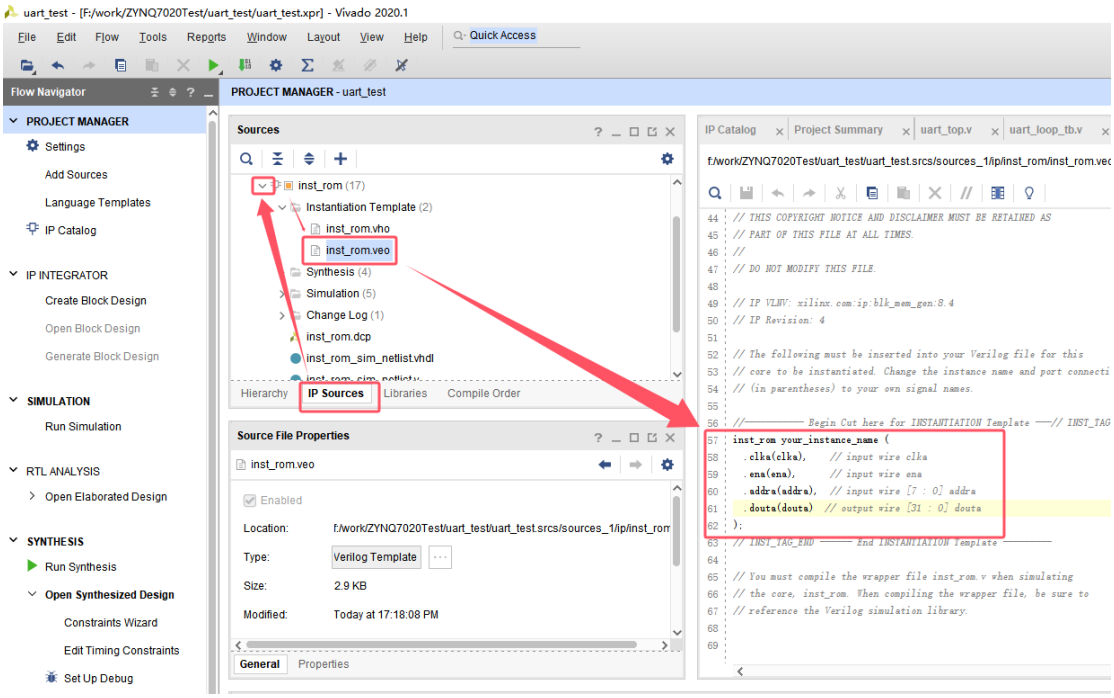
一个 COE 文件的样例如下：

```
memory_initialization_radix=16;
memory_initialization_vector=
00000108,
00000304,
00000506,
00000907,
00000001,
00000004,
00000005,
00000009,
00000007,
00000005,
00000000,
00000005,
00008888,
00008888,
00009999,
00009999,
0000Aaaa,
0000aaaa,
0000Bbbb,
0000bbbb,
00001234,
00005678,
000012ab,
0000cdef;
```

可以直接用文本编辑器写好 COE 文件，memory_initialization_radix=16;表示后面的数据为 16 进制，同理 radix=2 表示后面的数据是二进制。第二行为数据向量的起始行，照抄即可，第三行开始是数据（我们定义的 32 位数据，与 rom 数据宽度一致），数据间隔用逗号，最后一个数据用分号表示结束。每一行都代表 ROM 中的一个地址，在这个文件中，00000108 可以通过地址 0 来寻址，00000304 可以通过地址 1 来寻址，以此类推。

点击 ok，生成 rom ip 核相应的文件。需要注意，每次更新 COE 文件后都需要双击 IP 核重新选择文件进行生成。

接下来需要在自己的代码中例化这个 IP。在 source 下面有一个 IP Source 选项卡点击进去，再点击 Instantiation Template 里的.veo 文件就是 verilog 例化文件了。同路径中的.vho 文件为 VHDL 的例化文件。将 veo 中的模块声明拷贝到自己的代码中，修改例化的名称以及接口信号，开始实验。



2. 串口模块说明

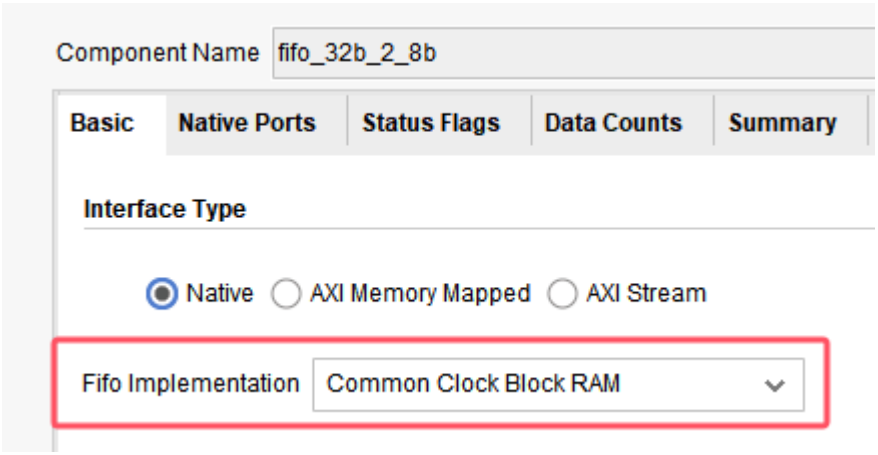
本次实验依然会提供串口模块。与之前的实验不同的是，本次提供的模块已经将 RX 与 TX 进行包装，仅暴露数据接口与握手信号，同学们只需要按照上次实验时的时序进行操作即可。

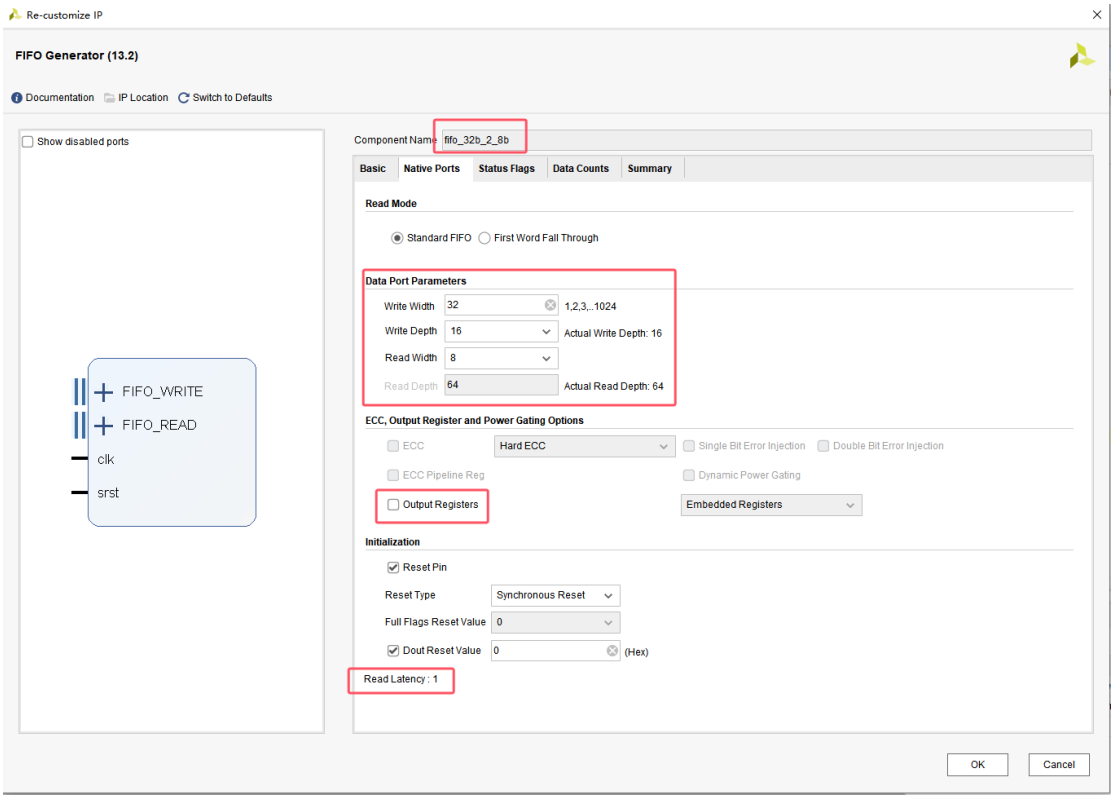
```
52  uart_top u_uart_top(
53      .clk_i      (clk_i      ),
54      .rst_n      (rst_n      ),
55      .rx_i       (rx_i       ),
56      .tx_o       (tx_o       ),
57      .tx_valid_i (tx_valid   ),
58      .tx_ready_o (tx_ready   ),
59      .tx_data_i  (tx_data    ),
60      .rx_valid_o (rx_valid   ),
61      .rx_ready_i (rx_ready   ),
62      .rx_data_i  (rx_data    )
63  );
```

与实验 3 的接口响应一致，当 valid 和 ready 同时为高电平时表示 data 有效。

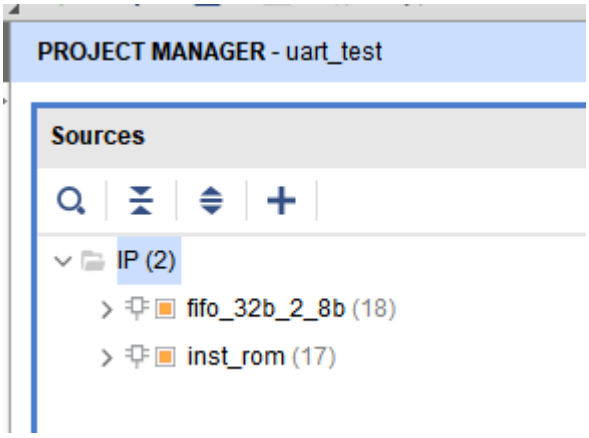
需要注意的是，本次实验中使用的 tx_data 端口位宽为 32bit，这是因为 rom 的长度也是 32bit，我们已经在 uart_top 中包装了将 32bit 转为串口 8bit 位宽的电路，具体设计可以参考附录，不需要你自己设计。为了使用这个模块，你还需要例化一个 FIFO，具体的配置项

如下图所示。



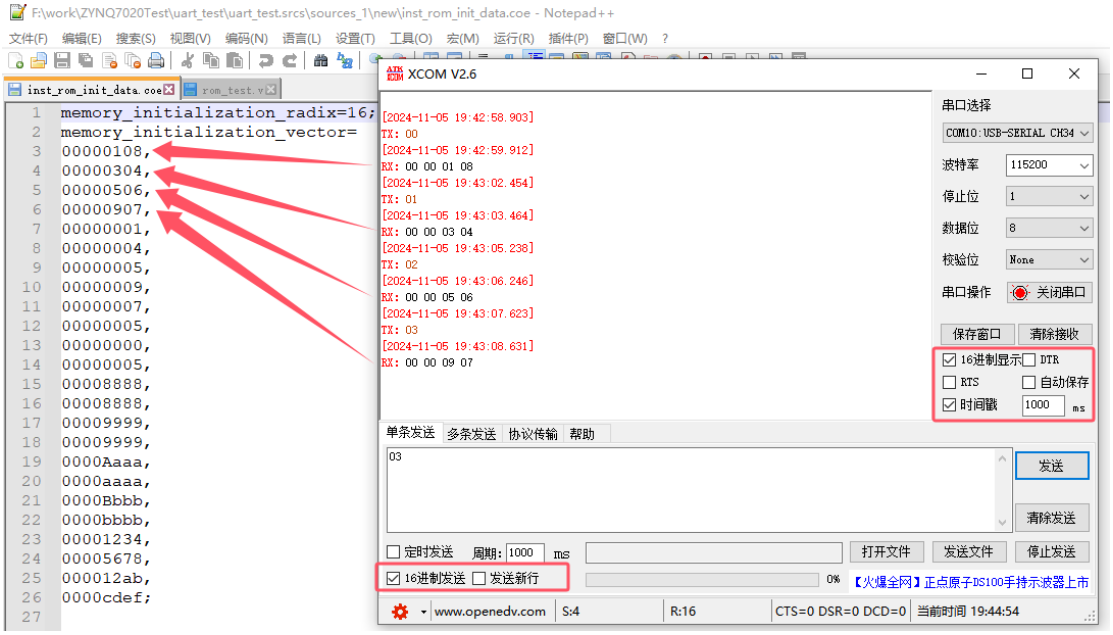


最后你的工程中应该只有这两个 ip。



五、 实验效果

我们本次实验需要达成的目标是在串口调试助手上发送一个地址，可以接收到 ROM 中这个地址的值。样例如下图所示，需要注意把 16 进制发送接收全部勾选上，并去掉发送新行的勾选。

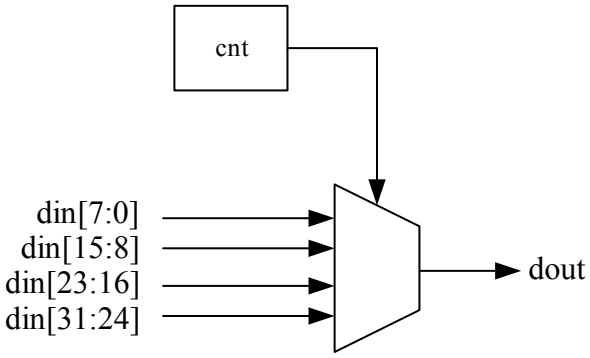


验收时，需要展示你的 coe 文件与串口调试助手的接收状态，数据中应有一行为你的学号。

如果遇到 bug，可以参考 FPGA 在线调试指南，自己尝试着进行 debug。

附录 1 串口模块内位宽转换电路说明

从 inst_rom 中取出的数据为 32bit，但串口一次只能发送 8bit 数据，我们需要将这 32bit 数据转换为 4 个 8bit 数据进行发送。一个简单的想法就是使用一个计数器，电路图绘制如下：



verilog 代码描述如下：

```
106   input        en;
107
108   wire [31:0] din;
109   wire [7:0]  dout;
110
111   reg    [1:0] cnt;
112
113   always @(posedge clk_i or nededge rst_n)
114       if(rst_n) begin
115           cnt <= 2'b00;
116       end else begin
117           if(en)
118               cnt <= cnt + 1'b1
119           end
120   end
121
122   always @(*)begin
123       case(cnt)
124           2'b00: dout = din[7:0];
125           2'b01: dout = din[15:8];
126           2'b10: dout = din[23:16];
127           2'b11: dout = din[31:24];
128       endcase
129   end
```

但 Vivado 已经给我们提供了一个非常简单的工具： Non-symmetric aspect ratios (different read and write data widths)，也就是输入输出位宽不一致的 FIFO。在 Xilinx 的文档中对这一行为的描述如下：

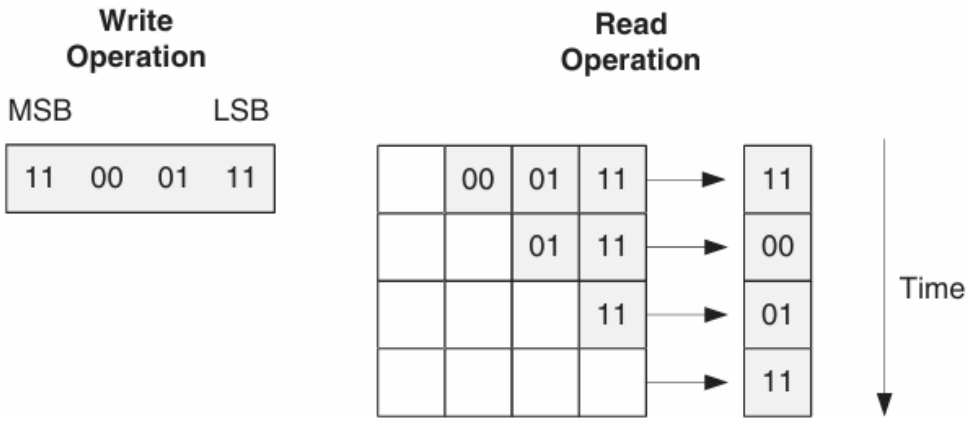
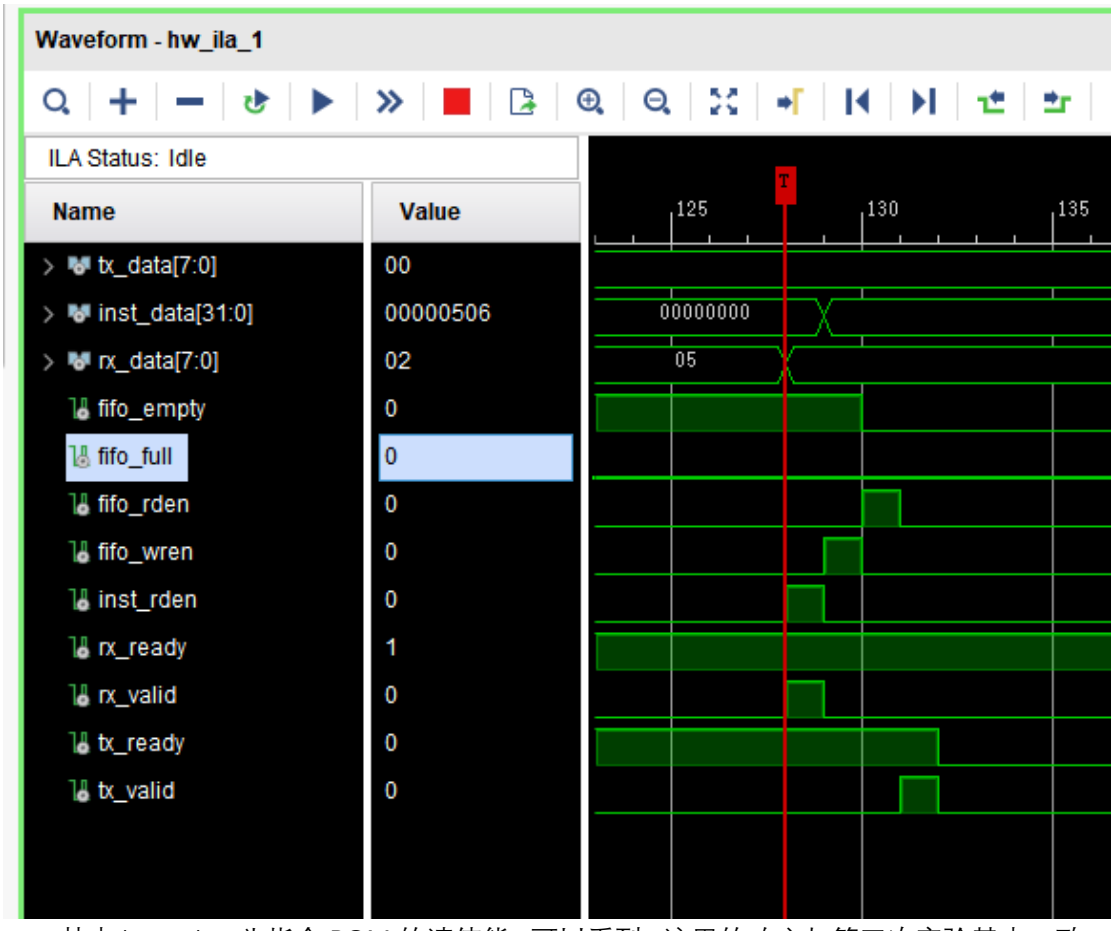
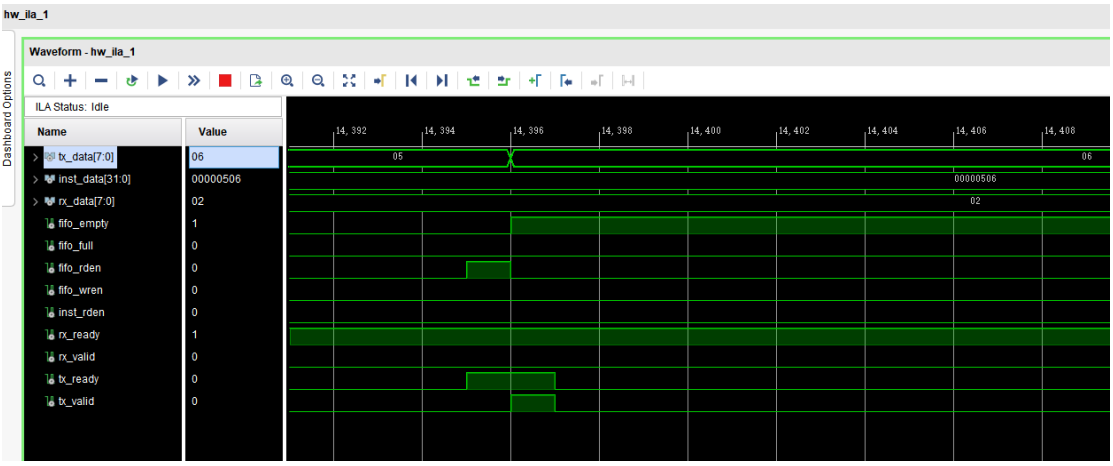


Figure 3-15: **4:1 Aspect Ratio: Data Ordering**

因此，我们只需要使用一个 FIFO 完成这一过程即可。剩下的就是控制握手信号，波形图如下：

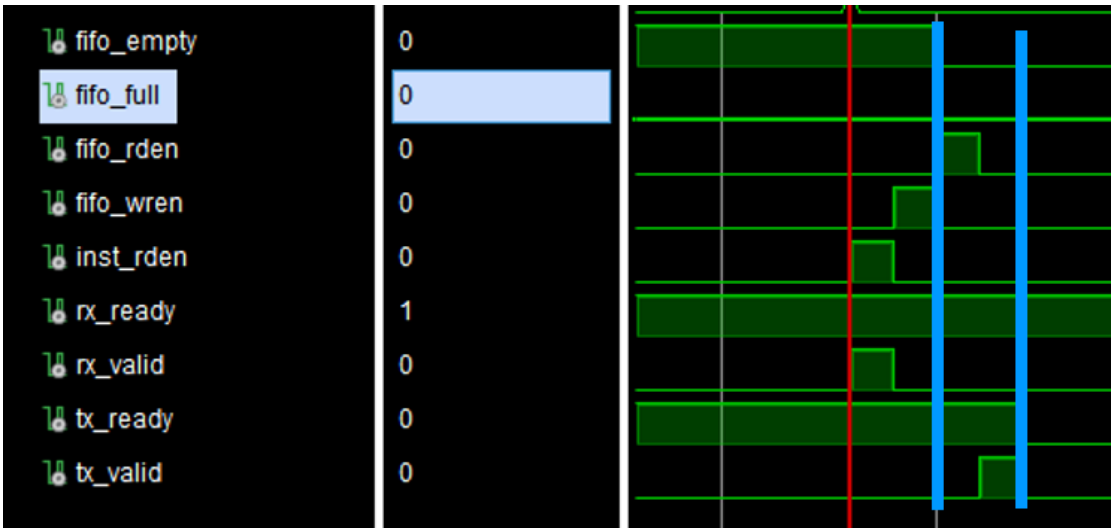


其中 inst_rden 为指令 ROM 的读使能，可以看到，这里的响应与第三次实验基本一致，只是在 fifo 的读使能拉高后，fifo_empty 依然没有拉高，这是因为 fifo 中还有三组数据。



在读完最后的数据发送到 tx_data 后，fifo_empty 才拉高，表示 FIFO 中所有的数据都已发送完毕。

这段电路在设计时会遇到一个问题：fifo_rden 如何控制。在第三次实验中，我们会通过判断 FIFO 非空且 TX 准备好发送下一组数据决定是否拉高 fifo_rden，但在这里，我们会发现 fifo_rden 会被拉高两拍，导致原本应该发送 4*8 的数据只能收到 2*8。



解决方法也比较简单：控制 fifo_rden 只能拉高一拍，具体方法请自己阅读代码学习。

附录 2 握手

做了两次实验，你是否在思考这个 valid 和 ready 到底是什么意思，本章节会给你一个比较简单的解释。

首先，所有的通信协议都要做到一件事：保证数据在传输过程中不会丢失。而在传输中，排除通信链路的故障，最容易出现的问题就是：接收端没有准备好接收数据而发送方将数据发出；接收端没有收到数据但发送方以为数据已经被接收。因此，我们需要有一个方法，让发送方与接收方能够知道对方的状态。

现在来看 valid 与 ready。valid 表示发送端已经准备好发送数据，ready 表示接收端已经准备好接收数据。当 valid 与 ready 同时为高时，代表着发送端与接收端都知道对方能够完成一次数据传输过程，这就是一次握手：双方都获取到了对方的状态。发送端和接收端内部可以通过监听这两根信号线的状态来控制内部逻辑工作。

但这还有一个问题：握手过程需要有一方主动发送 valid/ready 信号，另一方接收到后再启动握手过程。如果双方都主动发送，可能会导致内部逻辑没来得及跳转而丢失数据；如果双方都被动监听，则握手过程根本不会启动。这就需要在设计时有一方主动请求握手，另一方监听握手信号。一般我们将监听端称为主机(Master)，请求端称为从机(Slave)。从机需要主动发起握手过程，主机则根据自身情况决定与哪个从机启动握手。

现在，你应该对这个简单的握手协议有了基本的认识。我们在实验中使用的协议名称是 AXI-Stream，有兴趣可以自己学习一下，它来源于 AXI 协议，那是一个更加复杂、功能更完善的通信协议。